

Discriminative Project – Milestone 1 Report

Recognizing 73 Everyday Objects: Comparing Classification and Object Detection Models

Study Group 8: Pooja Mishra · Gaurang Patil · Sankalp Shinde · October 2026

Contributed objects: OBJ061 Card wallet (Pooja Mishra) · OBJ069 knife (Gaurang Patil) · OBJ059 Comb (Sankalp Shinde)

Summary. Four CNN-based models were trained and tested on the final merged class dataset of 73 Object IDs: a custom CNN trained from scratch, a pretrained EfficientNet-B0, and two object detectors (YOLO11s and Faster R-CNN). All four were evaluated on the same held-out test set of 1,098 images. **EfficientNet-B0 performed best, with 95.4% test accuracy and a macro-F1 of 0.953**, while also being among the fastest and smallest models. It is selected as the final model for this dataset.

1. Dataset and Problem Overview

Each student in the course photographed one unique everyday object, producing roughly 100 images per object. The images were merged into a shared dataset in which every object has an Object ID (OBJ001–OBJ073), for example OBJ001 Basketball, OBJ005 Computer mouse and OBJ009 Marshall Speaker. **The task is to identify which of the 73 objects appears in a photo**, a 73-class image classification problem. The two detection models additionally locate the object with a bounding box.

The dataset is challenging for two reasons. First, there are only about 70 training images per class. Second, several objects are visually close to one another, such as Sunglasses and Wayfarer Sunglasses; Watch, Analog watch and iWatch; three kinds of racket; and several wallets and plush toys.

1.1 Preprocessing

All preprocessing was done in code, following the TA-recommended workflow: image validation, exact-duplicate check, resizing, RGB conversion and a stratified split. The same cleaned dataset and split were used for all four models.

Step	What was done	Result
Object ID check	Dataset folders compared against the official Object ID list	73 / 73 IDs matched, none missing or extra
Validation	Every file opened and verified; dimensions and color mode recorded	7,343 images, 0 corrupt, all RGB
Duplicates	Exact duplicates found by MD5 file hash and removed before splitting (prevents train/test leakage)	28 removed (all within the same class); 7,315 unique images
Resizing	Images not 224×224 resized; non-square images letterboxed so the whole object is kept	3 images resized, none dropped
Class balance	Folders with more than 100 images kept in full (extra images can be valid)	94–113 images per class (median 100)
Split	Stratified 70 / 15 / 15 split, fixed random seed (42)	Train 5,120 · Validation 1,097 · Test 1,098

Table 1. Preprocessing steps and their effect on the dataset.

Normalization and augmentation were applied per model during training. The classifiers used ImageNet normalization with random resized crops, horizontal flips, $\pm 15^\circ$ rotations and color jitter. The detectors used flips and color jitter; YOLO11s also used its built-in augmentation pipeline. Validation and test images were never augmented.

1.2 Bounding boxes for the detection models

The dataset provides one Object ID label per image but **no bounding boxes**, which object detectors need for training. Boxes were therefore generated automatically with OWLv2, an open-vocabulary detector that can locate an object from a text description (for example, “a photo of a Basketball”), using the object names from the official list. Because every image shows a single object, the highest-scoring box was kept. When OWLv2 was not confident (score below 0.10, or an implausibly small box), the whole image was used as the box. **This fallback was needed for 13.6% of images.** These generated boxes are pseudo-labels rather than hand-drawn ground truth, which affects how the detection metrics in Section 3 should be read.

2. Models Tested

As suggested by the TAs, more than one approach was tested: a CNN trained from scratch, a pretrained CNN using transfer learning, and two CNN-based object detectors of different designs.

Model	Task	Approach	Parameters
Custom CNN	Classification	Trained from scratch: 5 blocks of [conv–BN–ReLU ×2 + max-pool] (32 → 256 filters), global average pooling, dropout 0.3	2.4 M
EfficientNet-B0	Classification	ImageNet-pretrained; new 73-class head; whole network fine-tuned	4.1 M
YOLO11s	Detection	One-stage detector (Ultralytics), COCO-pretrained, fine-tuned	9.5 M
Faster R-CNN (ResNet50-FPN v2)	Detection	Two-stage detector (region proposals + classification), COCO-pretrained, fine-tuned	43.6 M

Table 2. The four models compared.

Training setup. The classifiers were trained with AdamW, a one-cycle learning-rate schedule, label smoothing (0.1) and mixed precision. EfficientNet-B0 used a lower learning rate for the pretrained backbone (2e-4) than for the new head (2e-3). The custom CNN was allowed up to 40 epochs and EfficientNet-B0 up to 15, with early stopping on validation macro-F1. YOLO11s trained at 224×224 for up to 60 epochs (patience 15). Faster R-CNN trained for 12 epochs with SGD (learning rate 0.01, warm-up and cosine decay) at an input size of 320. For every model, the checkpoint with the best validation score was kept, and the test set was used only once, for the final evaluation. All training ran on a single NVIDIA T4 GPU.

3. Experimental Results

All models were scored on the same 1,098 test images against the true Object ID labels. For the detectors, an image's prediction is the class of its most confident detection. Macro-averaged metrics weight all 73 objects equally.

Model	Accuracy	Macro precision	Macro recall	Macro-F1	Top-5 acc.	Train time	ms / image
Custom CNN	89.6%	0.911	0.896	0.897	96.4%	30.2 min	0.88
EfficientNet-B0	95.4%	0.957	0.953	0.953	98.2%	11.6 min	0.75
YOLO11s	93.6%	0.940	0.936	0.936	–	27.4 min	5.3
Faster R-CNN	88.9%	0.898	0.889	0.888	–	68.1 min	30.7

Table 3. Test-set results for all four models (best values in bold).

Timing note: for the classifiers, ms/image is the network's forward pass in batches of 64. For the detectors, it is the full prediction pipeline including pre- and post-processing, so the detector times are somewhat inflated relative to the classifiers.

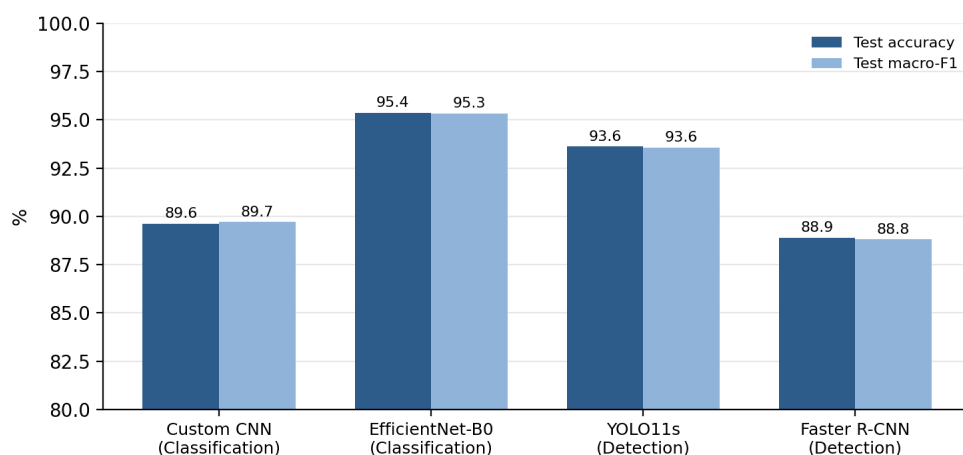


Figure 1. Test accuracy and macro-F1 of all four models.

Detector	mAP@0.5	mAP@0.5:0.95	Mean IoU of top box
YOLO11s	0.831	0.770	0.837
Faster R-CNN	0.824	0.720	0.820

Table 4. Localization metrics (COCO-style mAP), measured against the automatically generated boxes.

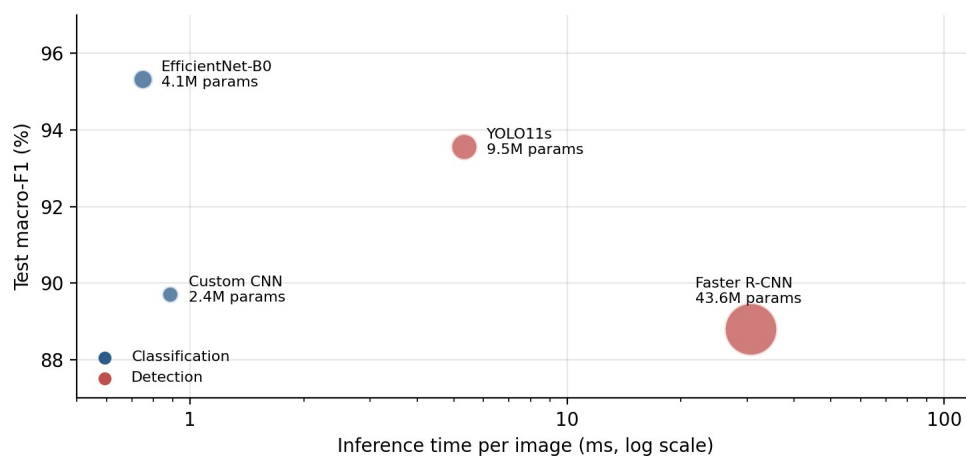


Figure 2. Accuracy vs. inference cost. Bubble size is proportional to the number of parameters. EfficientNet-B0 sits in the best corner: highest F1 at the lowest cost.

4. Comparison and Analysis

4.1 Transfer learning vs. training from scratch

The pretrained EfficientNet-B0 beat the custom CNN by **5.7 percentage points in accuracy** (95.4% vs. 89.6%), and trained in less than half the time (11.6 vs. 30.2 minutes). With only about 70 training images per object, a network trained from scratch must learn edges, textures and shapes from very little data. EfficientNet-B0 starts with features already learned from over a million ImageNet images and only needs to adapt them to these 73 objects. The custom CNN still did well for its size: 89.6% accuracy and 96.4% top-5 accuracy with just 2.4 M parameters. It also used all 40 epochs without triggering early stopping, which suggests it had not fully converged and could improve with longer training.

4.2 Classification vs. detection

YOLO11s reached 93.6% accuracy, only 1.7 points behind EfficientNet-B0, while also outputting where the object is. On this dataset, however, the localization does not help with recognition. Each photo contains one dominant object, so a classifier that looks at the whole image already has all the information it needs. The detectors add cost without adding accuracy: they need bounding-box labels, have more parameters, and are slower at inference.

4.3 One-stage vs. two-stage detector

YOLO11s outperformed Faster R-CNN on every measure: **4.7 points higher accuracy, a higher mAP@0.5:0.95 (0.770 vs. 0.720)**, about 4.6× fewer parameters, roughly 6× faster inference and 2.5× shorter training. The two detectors found the object about equally well at a loose overlap threshold (mAP@0.5 of 0.831 vs. 0.824). YOLO11s produced tighter boxes, which shows up at the stricter thresholds of mAP@0.5:0.95. Several factors favor YOLO here:

- Faster R-CNN's region-proposal stage is designed for cluttered scenes containing many objects, which this single-object dataset does not need.
- Faster R-CNN was limited to 12 epochs by its much higher compute cost, while YOLO11s trained for up to 60.
- Upscaling the 224×224 images to 320 for Faster R-CNN adds compute but no new detail.

4.4 Error analysis

Precision was slightly higher than recall for every model, but the gap is small (at most 1.5 points), which is consistent with the near-balanced classes. Table 5 lists the five weakest Object IDs for each classifier.

Rank	Custom CNN – weakest Object IDs	F1	EfficientNet-B0 – weakest Object IDs	F1
1	OBJ061 · Card wallet	0.45	OBJ013 · Mobile phone	0.76
2	OBJ013 · Mobile phone	0.50	OBJ061 · Card wallet	0.84
3	OBJ034 · Table Tennis Racket	0.52	OBJ034 · Table Tennis Racket	0.85
4	OBJ005 · Computer mouse	0.72	OBJ023 · Bracelet	0.88
5	OBJ045 · Wayfarer Sunglasses	0.79	OBJ024 · Plush doll	0.88

Table 5. The five weakest Object IDs per classifier (test-set F1; about 15 test images per Object ID).

The same three objects are the hardest for both models: OBJ013 Mobile phone, OBJ061 Card wallet and OBJ034 Table Tennis Racket. All three are small, flat, mostly plain objects whose outline (a dark rectangle or a paddle shape) resembles other items in the dataset. In the custom CNN, OBJ061 Card wallet acted as a catch-all for such objects: its precision was only 0.36, because Combs (OBJ059), Purses (OBJ054) and Mobile phones (OBJ013) were all predicted as Card wallet. OBJ059 Comb → OBJ061 Card wallet, two objects contributed by Study Group 8 members, is the only confusion that both models made more than once (2 errors each).

The custom CNN's errors suggest it relies on low-level cues such as color, texture or setting, rather than shape. Examples include OBJ057 Power Bank → OBJ001 Basketball, OBJ027 Tennis ball → OBJ023 Bracelet, OBJ071 Sloth Plushie → OBJ008 Cat, and OBJ005 Computer mouse → OBJ012 Computer Keyboard (two computer accessories likely photographed in similar desk settings). This fits a network that learned its features from only about 70 images per class.

EfficientNet-B0's errors are rare and scattered. Apart from Comb → Card wallet, no pair of objects was confused more than once, and the only mix-up between objects of the same kind was OBJ035 Badminton racket → OBJ049 Pickleball bat. Its other errors, such as OBJ026 Red Sox Cap → OBJ048

Wireless powerbank, look like individual difficult photos rather than a systematic weakness. Notably, the look-alike groups listed in Section 1 caused few problems. The three watches did not appear among either model's weakest classes or most-confused pairs, and OBJ045 Wayfarer Sunglasses was one of the custom CNN's weakest classes but not EfficientNet-B0's. This shows that pretrained features separate fine-grained differences well.

4.5 Limitations

- **Detection metrics use pseudo-labels.** mAP and IoU measure agreement with the automatically generated boxes, not with hand-drawn ground truth. For the 13.6% of images with a full-image fallback box, the detectors were also trained to predict a full-image box. Image-level accuracy is unaffected, because it is scored against the true Object IDs.
- **Near-duplicates.** Only exact duplicates were removed. Near-identical photos of the same object (for example, burst shots) may still appear in both the training and test sets, which could slightly inflate all four models' scores.
- **Single split.** Results come from one stratified split with one random seed. With about 15 test images per object, a single misclassified photo changes that object's recall by about 7 points, so per-class scores are noisy, although the overall comparison is reliable.

5. Final Model Selection and Justification

EfficientNet-B0 is selected as the most suitable model for this dataset.

- **Most accurate:** highest test accuracy (95.4%), macro-F1 (0.953) and top-5 accuracy (98.2%) of all four models. The balanced precision and recall (0.957 / 0.953) show that it performs consistently across Object IDs.
- **Chosen without bias:** it was selected on validation macro-F1 before the test set was consulted, and the test results confirmed the choice.
- **Efficient:** only 4.1 M parameters, the shortest training time (11.6 minutes) and the fastest inference (0.75 ms per image) of the four models.
- **Fits the problem:** each photo contains a single object, so whole-image classification is the right formulation. Detection adds labelling and compute cost without improving recognition.

If an application also needs to know where the object is in the image, **YOLO11s is the recommended alternative**. It offers 93.6% accuracy with good localization at a moderate cost. For future milestones, the most useful next steps are hand-annotating a small set of bounding boxes to measure true detection accuracy, training the custom CNN for longer, and repeating the experiments with several random seeds.